

面向实时点对点通信的 NAT 穿越机制研究——STUN、TURN 与 ICE 的协议协同及路径选择

1120231863 左逸龙

北京理工大学 计算机学院 07112303 班, 北京 100081

cs.yilong.zuo@bit.edu.cn

NAT Traversal for Real-Time Peer-to-Peer Communication: Protocol Coordination and Path Selection with STUN, TURN, and ICE

Yilong Zuo

(Class 07112303, School of Computer Science, Beijing Institute of Technology, Beijing 100081)

Abstract Network address translation (NAT) permits many private hosts to share limited public IPv4 addresses, but it also removes the stable, globally reachable transport endpoint assumed by conventional peer-to-peer communication. This paper studies how real-time applications recover reachability through the coordinated use of Session Traversal Utilities for NAT (STUN), Traversal Using Relays around NAT (TURN), and Interactive Connectivity Establishment (ICE). The analysis begins with NAT mapping and filtering behavior, then explains UDP hole punching and the conditions under which a direct path can be established. STUN is examined as a mechanism for discovering server-reflexive addresses and performing connectivity checks, while TURN is treated as a reliability path that allocates relay addresses and forwards application traffic. ICE combines host, server-reflexive, peer-reflexive, and relayed candidates into prioritized candidate pairs, checks them with STUN transactions, and nominates a selected pair. A WebRTC scenario is used to connect these protocol roles with signaling, Trickle ICE, and media transmission. The paper also compares direct and relayed paths in terms of latency, reachability, server bandwidth, security, and address privacy. A lightweight browser experiment is designed to compare candidate gathering with no ICE server, a STUN server, and a STUN/TURN configuration. The resulting framework prepares and evaluates direct and relayed possibilities together, and the selected path represents a practical balance among performance, connection reliability, operating cost, and privacy.

Key words network address translation; NAT traversal; STUN; TURN; ICE; WebRTC; peer-to-peer communication

摘要 网络地址转换允许多个私网终端共享有限的公网 IPv4 地址,但也使传统点对点通信失去稳定、可直接访问的传输端点。本文围绕实时点对点通信的连接建立过程,分析 NAT 映射与过滤行为、UDP 打洞以及 STUN、TURN 和 ICE 的协议协同。STUN 用于获得服务器反射地址并支持连通性检查,TURN 在直接路径不可用时提供中继地址和数据转发,ICE 则负责收集多类候选、构造候选对、执行检查并提名最终路径。本文进一步以 WebRTC 为例说明信令、Trickle ICE 与媒体路径之间的关系,并从时延、成功率、服务器成本、安全和隐私角度比较直连与中继。本文还设计浏览器候选收集实验,分别考察无 ICE 服务器、仅 STUN 和 STUN 加 TURN 三种配置。分析表明,实际 NAT 穿越会同时准备多种可能路径,再依据检查结果和优先级选择可用路径;工程实现需要在直连性能与中继可靠性之间取得平衡。

关键词 网络地址转换; NAT 穿越; STUN; TURN; ICE; WebRTC; 点对点通信

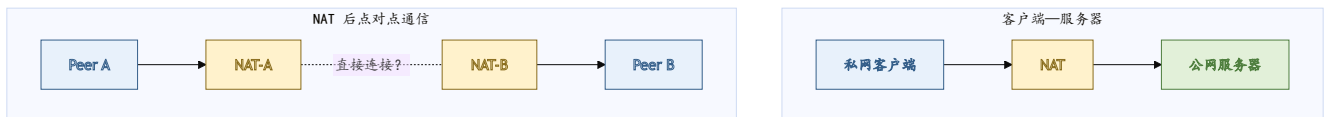


图1 客户端—服务器与 NAT 后点对点通信的差异
Client-server and peer-to-peer communication behind NATs

1 引言

互联网应用早期通常假设通信端点拥有全局唯一且可路由的 IP 地址。随着终端数量增长,IPv4 地址不足推动家庭、校园和企业网络广泛部署网络地址转换 (Network Address Translation, NAT)。NAT 将私网地址和端口转换为少量公网端点,使多个内部终端能够共享公网地址。该机制降低了公网 IPv4 地址的消耗,也改变了端到端通信的可达性条件:内网终端可以先向公网服务器发送数据并建立映射,公网中的其他主机却难以在没有映射状态时主动访问该终端。

客户端—服务器应用通常由客户端主动连接具有稳定公网地址的服务器,因此能够自然触发 NAT 建立映射。点对点 (Peer-to-Peer, P2P) 通信面临的条件更严格。当通信双方均位于 NAT 或状态防火墙之后时,双方交换到的私网地址不能在互联网中路由;即使获得各自的公网映射地址,该映射能否复用、入站数据能否通过过滤,也取决于两侧设备的具体行为。图1对比了两种通信结构。左侧路径由私网客户

端主动发起,右侧两个端点都缺少可直接使用的远端地址,因此需要额外的地址发现、同步发送和备用中继机制。

本文围绕一个完整的连接建立问题展开:两个 NAT 后端点如何发现可能使用的地址,如何验证直接路径,以及直连失败时如何借助中继继续通信。分析内容包括 NAT 的映射和过滤行为、UDP 打洞、STUN、TURN、ICE 以及 WebRTC 中的实际组合方式。讨论重点放在 UDP 实时通信;TCP 打洞、运营商级 NAT 和 IPv6 只作必要说明。全文先解释 NAT 行为和打洞条件,再分析三种协议的职责与协同,随后放入 WebRTC 场景,并通过工程比较和轻量实验说明路径选择的实际含义。

2 NAT 工作原理及行为特征

2.1 地址与端口转换

传输层通信端点通常写成 IP:port。IP 地址定位主机或网络接口，端口号区分主机上的不同通信进程。以 UDP 为例，内部主机 192.168.1.10:5000 向公网服务器 198.51.100.20:3478 发送数据时，NAT 可以把数据包源端点改写为 203.0.113.8:62001，并在映射表中保存内部端点、公网端点、传输协议及必要的远端信息。返回数据到达公网端口 62001 后，NAT 查询映射表并恢复内部目的地址。映射具有生命周期；如果端点长期不发送数据，设备可能回收表项，后续入站数据也就失去转发依据。

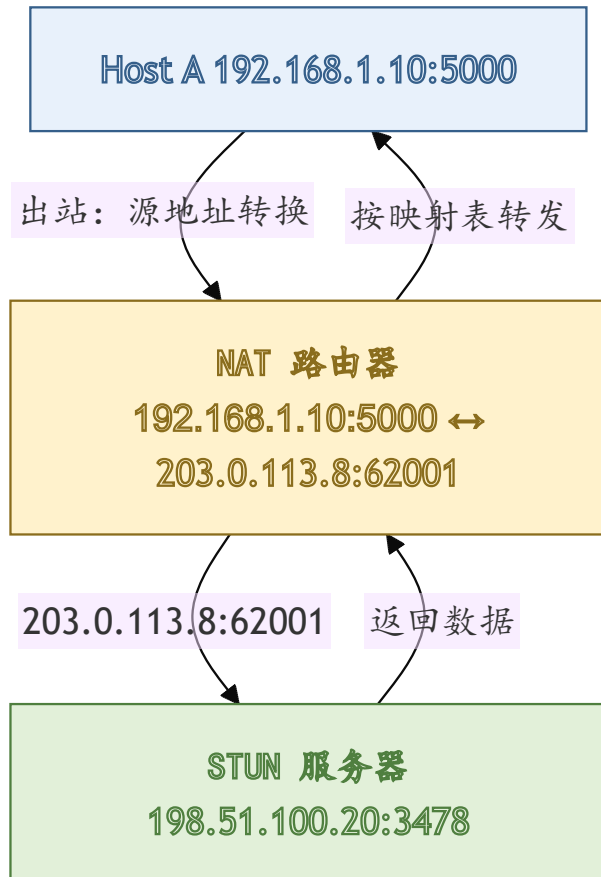


图2 NAT 地址与端口转换过程

图2中的两个方向说明 NAT 不只是修改出站数据包，还要依靠已建立的状态处理返回流量。公网端点属于 NAT 的外部表现，不等同于内部主机固定拥有的地址。另一个内部端点或另一种传输协议通常会形成不同表项，因此讨论 NAT 穿越时必须同时给出 IP、端口和协议。

2.2 映射行为

RFC 4787 将 UDP 映射行为按远端依赖程度分开描述 [1]。端点无关映射 (Endpoint-Independent Mapping, EIM) 在同一内部端点访问不同远端时复用同一公网端点。地址相关映射 (Address-Dependent Mapping, ADM) 会在远端 IP 改变时建立另一映射，但访问同一 IP 的不同端口仍可能复用。地址和端口相关映射 (Address-and-Port-Dependent Mapping, APDM) 同时依赖远端 IP 与端口，同一内部端点访问不同远端端口也可能得到不同公网端口。

映射行为	受远端 IP 影响	受远端端口影响	对打洞的影响
端点无关映射	否	否	公网映射较易复用
地址相关映射	是	否	需要考虑对端地址
地址和端口相关映射	是	是	对服务器观察值的复用较困难

表1 NAT 映射行为比较

表1中“受远端影响”描述的是公网映射如何产生。UDP 打洞希望协调服务器观察到的公网端点也能用于访问另一个 peer，EIM 最符合这一条件。ADM 和 APDM 下，端点转向 peer 发送时可能得到新映射，服务器先前观察到的地址随之失效。不过，映射行为只能说明地址是否复用，不能单独判断返回数据是否允许进入。

2.3 过滤行为

过滤行为决定已存在映射上的入站数据由哪些外部端点发送时可以通过。端点无关过滤 (Endpoint-Independent Filtering, EIF) 允许任意外部端点向已映射公网端点发送。地址相关过滤 (Address-Dependent Filtering, ADF) 要求内部主机曾向该外部 IP 发送。地址和端口相关过滤 (Address-and-Port-Dependent Filtering, APDF) 进一步要求外部 IP 和端口均与出站目标一致 [1]。

过滤行为	外部数据进入条件	穿越难度
端点无关过滤	映射存在即可接收入站数据	较低
地址相关过滤	内部端点曾向该外部 IP 发送	中等
地址和端口相关过滤	内部端点曾向相同外部 IP 和端口发送	较高

表2 NAT 过滤行为比较

表2描述的是映射建立后的访问控制条件。一个 NAT 可以采用 EIM，同时采用严格的 APDF；此时服务器看到的公网端点可能保持不变，但 peer 的首个入站数据仍会被丢弃。双方近似同时向对方发送的意义正在于此：各自的出站数据为 NAT 建立或刷新映射，并满足针对对端的过滤条件。现实设备还可能同时部署独立防火墙，NAT 负责地址转换，防火墙负责访问控制，两者的效果不能简单合并为一种行为。

2.4 传统 NAT 类型与行为描述

早期资料常使用 Full Cone、Restricted Cone、Port-Restricted Cone 和 Symmetric NAT。该分类便于直观介绍，却把映射与过滤绑定为固定组合，难以覆盖实际设备中的混合行为。本文采用 RFC 4787 的分离描述，以映射复用条件和入站允许条件分别判断穿越可能性。传统的 Symmetric NAT 大致对应远端相关的映射行为和较严格过滤，但不同设备仍可能在端口分配、超时与 hairpinning 上表现不同。Hairpinning 指同一 NAT 后主机通过 NAT 的外部映射互相通信，它对同网关内端点使用公网候选时很重要 [1]。

3 点对点通信中的 NAT 穿越

3.1 公网协调服务器

设 Peer A 位于 NAT-A 后，Peer B 位于 NAT-B 后。双方的私网地址只在各自网络内有效，因此需要一个具有公网地址的协调服务器 (Rendezvous Server)。A 和

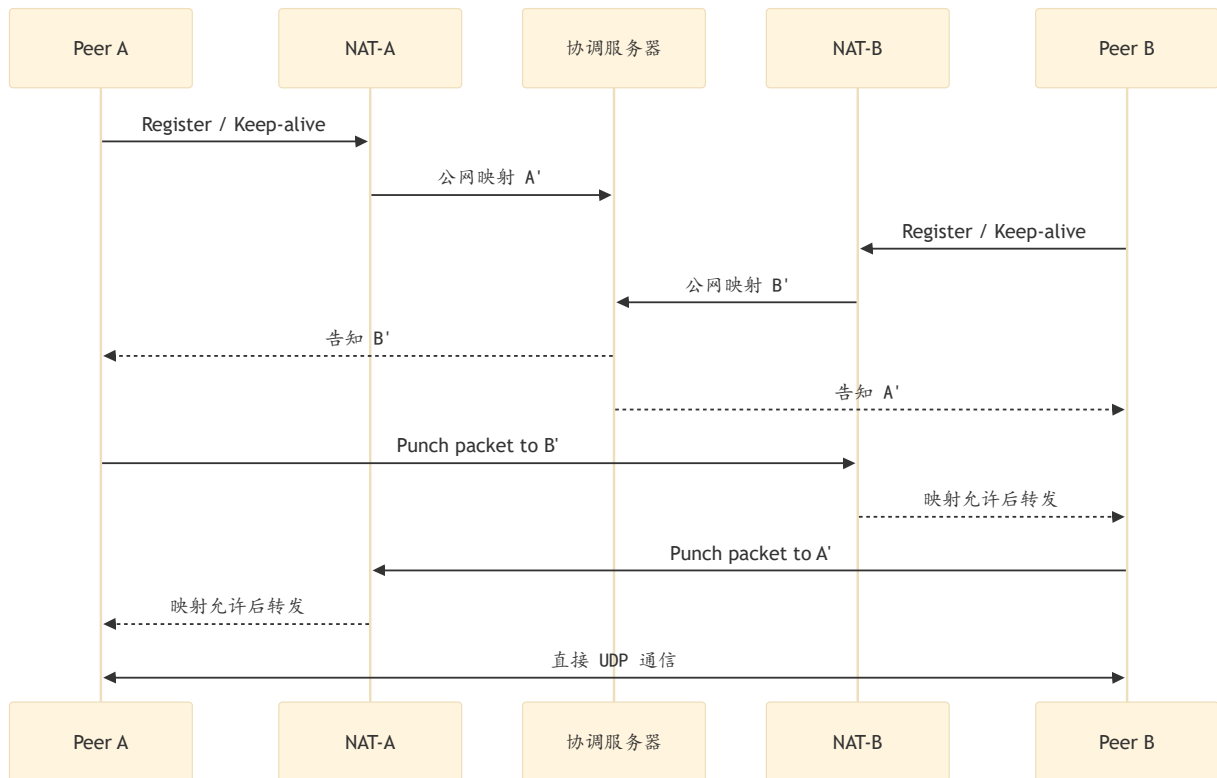


图3 基于公网协调服务器的UDP打洞流程
UDP hole punching with a public rendezvous server

B 分别主动连接服务器，服务器可以观察两者数据包的公网源端点，并通过已有会话交换连接参数。协调服务器主要承担发现、信令和同步作用，最终业务数据是否经过服务器取决于所选路径；若打洞成功，媒体数据可以绕过该服务器直接传输 [2]。

3.2 UDP 打洞过程

UDP 打洞利用 NAT 对出站数据建立映射和过滤状态。A、B 先向协调服务器注册并保持映射。服务器把 A 的公网映射告知 B，也把 B 的公网映射告知 A。随后双方都向对端公网端点发送探测包。早到的一侧数据可能因另一侧尚未放行而丢失，但该数据已经在本侧 NAT 上建立面向对端的状态；当另一侧也发送后，后续数据就可能双向通过。经典研究和 RFC 5128 均将这种协同发送作为 UDP 点对点连接的基本方法 [2], [3]。图3的时间顺序包含两个不同阶段。注册消息让服务器获得公网映射，Punch packet 则在对端方向上建立 NAT 状态。图中首个探测包成功与否不是判断打洞失败的充分条件，应用通常需要重传和短时保活。建立直接路径后，映射仍可能因空闲超时消失，实时应用因此会周期性发送检查或保活数据。

3.3 失败原因与 TCP 打洞

打洞能否成功取决于两侧条件的组合。若 NAT 面向协调服务器和 peer 分配不同公网端口，服务器提供的地址可能无法复用；若过滤严格而双方发送时序差距过大，探测包会被持续丢弃。企业防火墙可能直接阻止 UDP，多层 NAT 和运营商级 NAT 还会增加状态链条。无线网络切换、映射超时和端口重分配也会使已交换候选过期。因而，公网反射地址的获得只证明

服务器能够观察该映射，不能证明任意 peer 都能使用它。TCP 打洞同样尝试让双方主动创建兼容状态，但 TCP 需要完成 SYN、SYN-ACK 和 ACK 握手，操作系统还要处理同时打开、监听端口和连接状态。NAT 对 TCP 状态的检查通常比 UDP 更复杂，端口预测也更依赖设备实现 [2]。实时音视频常以 UDP 为主要传输方式，本文后续集中讨论 UDP 上的 STUN、TURN 和 ICE；TCP 或 TLS 承载的 TURN 可作为受限网络中的补充路径。

4 STUN、TURN 与 ICE 协议机制

4.1 STUN：地址发现与连通性检查

会话穿越 NAT 实用工具（Session Traversal Utilities for NAT, STUN）是一种请求—响应协议。客户端向 STUN 服务器发送 Binding Request，服务器从收到的数据包中读取源 IP 和端口，再在 Binding Response 的 XOR-MAPPED-ADDRESS 属性中返回观察结果 [4]。位于 NAT 后的客户端据此获得服务器反射地址（server-reflexive address），ICE 可将其表示为服务器反射候选。异或编码的目的主要是避免某些应用层网关误改报文中的直写地址，它不提供加密保护。

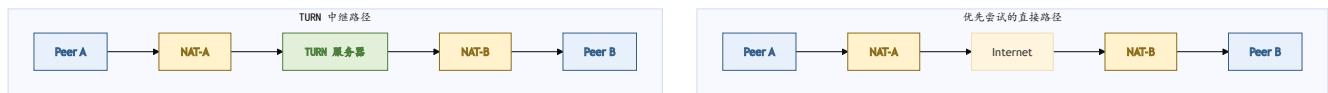


图 5 点对点直连路径与 TURN 中继路径
Direct peer-to-peer and TURN-relayed paths

候选类型	获取方式	典型适用范围	主要特点
主机候选	本地网络接口	同一局域网或公网接口	路径最直接
服务器反射候选	STUN 或 TURN 映射响应	NAT 后端点直连	表示服务器观察的映射
中继候选	TURN Allocation	严格 NAT 或防火墙环境	兼容性较强但占用服务器
对端反射候选	连通性检查中发现	实际源地址与已交换候选不一致	检查阶段动态产生

表 3 ICE 候选类型比较
Comparison of ICE candidate types

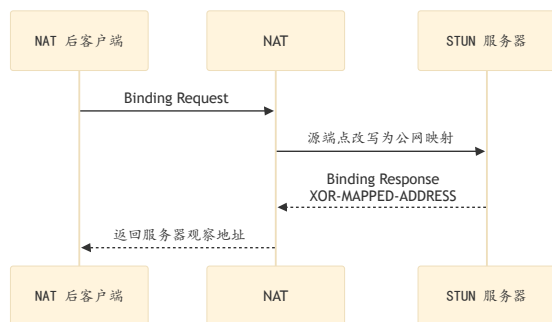


图 4 STUN Binding 交互与公网映射地址发现
STUN Binding exchange and mapped-address discovery

图 4 只涉及控制报文。STUN 服务器不会在两个 peer 之间持续转发应用数据；在 ICE 中，端点还使用带有认证信息的 STUN Binding 事务检查候选对是否可达。周期性检查能够刷新部分 NAT 映射，但保活间隔仍需结合网络行为设置。STUN 的局限来自观察位置：服务器看到的是客户端访问该服务器时的映射。如果 NAT 根据远端改变映射，或者防火墙不允许 peer 的入站 UDP，该反射地址仍可能无法形成直接路径。

4.2 TURN：中继通信机制

使用中继穿越 NAT (Traversal Using Relays around NAT, TURN) 在直接通信条件不足时提供受控中继。客户端向 TURN 服务器发送 Allocate Request，通过长期凭据认证后获得一个中继传输地址 (relayed transport address) 和具有生命周期的 Allocation [5]。客户端需要在到期前发送 Refresh。外部 peer 把数据发往中继地址，TURN 服务器根据 Allocation 将数据转给客户；反方向数据也由服务器转发，因此业务流量会持续占用服务器上下行带宽。

TURN 通过 Permission 限制哪些 peer IP 可以经某个 Allocation 通信，避免中继被任意主机利用。客户端可进一步使用 ChannelBind 把 peer 地址和 channel number 绑定，随后用 ChannelData 减少 Send Indication 的部分封装开销。Permission 和 ChannelBind 都有独立生命周期，需要按协议刷新。服务器还应实施认证、配额、速率限制和日志审计，公网部署不能配置为开放代理。

图 5 中，直连路径的业务数据不经过应用控制的中继服务器，中继路径则增加一个稳定的公网转发点。后者对 NAT 映射复用的要求较低，能够提高受限网络中的可达性，但它不能消除所有故障：客户端仍需访问 TURN 服务器，服务器位置、容量或故障都会

影响通信。实时视频的流量较大，中继路径还会带来明显的带宽成本。

4.3 ICE：候选组织与路径选择

交互式连接建立 (Interactive Connectivity Establishment, ICE) 是运行在通信端点上的候选收集、检查和选择机制，不对应一种独立服务器。ICE agent 从本地接口获得主机候选 (host candidate)，通过 STUN 或 TURN 的映射响应形成服务器反射候选，通过 TURN Allocation 获得中继候选 (relayed candidate)。连通性检查过程中，如果请求从尚未列出的地址到达，还可能产生对端反射候选 (peer-reflexive candidate) [6]。

双方通过应用层信令交换候选地址、候选类型、优先级、基础地址以及 ICE username fragment 和 password。ICE 只规定需要交换哪些连接参数，不规定信令必须采用 WebSocket、HTTP、SIP 或其他协议。每个本地候选与兼容的远端候选组成候选对 (candidate pair)。端点依据候选优先级和角色计算候选对优先级，形成检查清单 (checklist)，再用 STUN Binding Request 执行连通性检查。

候选对在检查过程中可经历 Frozen、Waiting、In-Progress、Succeeded 和 Failed 状态。一个端点担任 controlling agent，另一个担任 controlled agent；发生角色冲突时根据 tie-breaker 调整。成功检查证明该候选对在当前方向上可传输检查报文，ICE 还通过触发检查和响应机制建立双向可达状态。controlling agent 对成功候选对进行提名 (nomination)，被提名并确认的候选对成为选定候选对 (selected pair)，应用数据随后沿该路径传输 [6]。

候选优先级由 type preference、local preference 和 component 等因素共同构成。类型偏好通常使 host 高于 server-reflexive，使 relay 保持较低优先级；本地偏好则可以表达接口、地址族或网络成本差异。候选还带有 foundation，具有相同来源特征的候选可被归入同一基础组，ICE 借此控制检查解冻顺序并减少重复尝试。候选对排序提供的是检查次序，不是对实际时延的直接测量。高优先级候选对失败后，checklist 仍会继续处理其他可行组合，因此网络中存在中继候选不会阻止端点优先验证直接路径。

连通性检查还需要控制发送节奏。若端点同时对大量候选对快速发送 STUN 请求，容易在接入链路和 NAT 上形成短时突发。ICE 通过有序 checklist、事务重传和 triggered check 协调检查。Triggered check 可让收到有效请求的一侧尽快检查对应反向候选对，缩

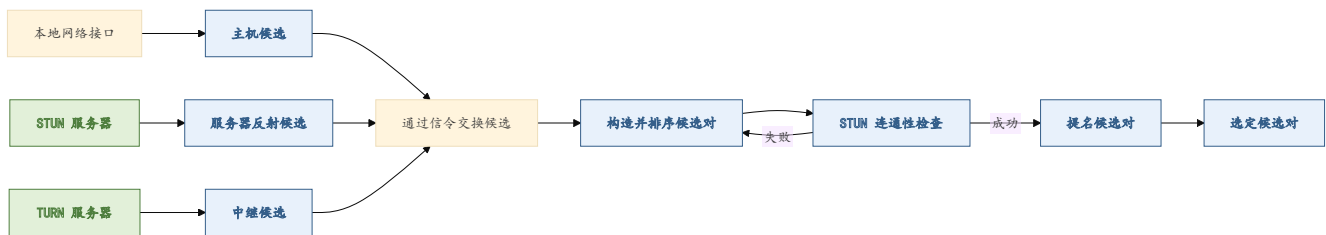


图 6 ICE 候选收集、连通性检查与路径选择
ICE candidate gathering, connectivity checks, and path selection

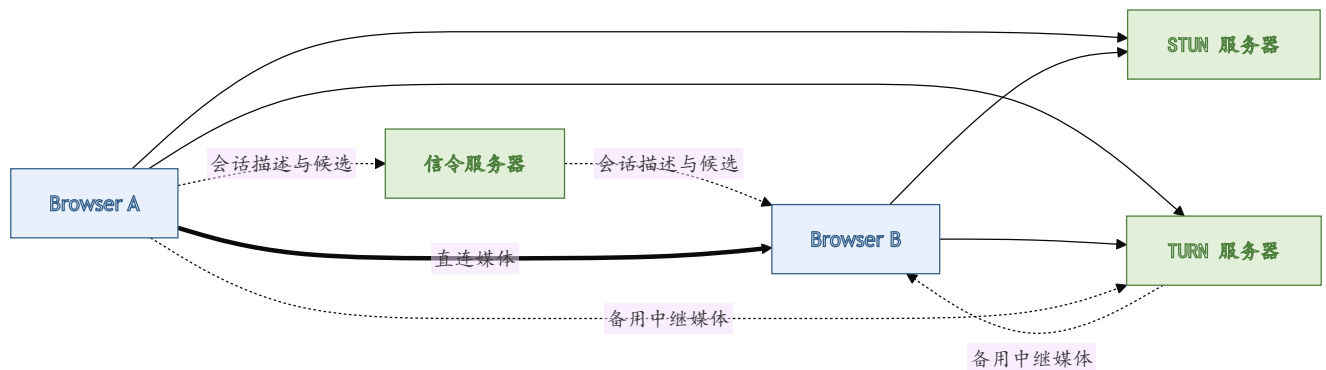


图 7 WebRTC 中信令、候选收集与媒体路径
Signaling, candidate gathering, and media paths in WebRTC

短双方各自按普通队列等待的时间。提名阶段则把“检查成功”进一步收敛为应用采用的路径；未被提名的成功候选仍可作为连接建立过程中的备选结果，而不会自动承载业务数据 [6]。

图 6 体现了 ICE 的核心顺序：候选先被收集和交换，随后形成多个候选对并接受检查，最终只提名其中一条或一组可用路径。TURN 候选可以在收集阶段提前产生，通常因类型优先级较低而排在直接候选之后。将过程概括为“STUN 失败后再启动 TURN”会遗漏候选并行准备和候选对排序，也无法解释 Trickle ICE 中边收集边检查的行为。

4.4 三者的协同关系

协议或机制	主要作用	转发业务数据	产物或结果
STUN	发现反射地址、执行连通性检查	否	映射地址和检查结果
TURN	分配中继地址并转发流量	是	中继候选与中继路径
ICE	组织候选、检查、提名和选择	否	选定候选对

表 4 STUN、TURN 与 ICE 的职责比较

Roles of STUN, TURN, and ICE

表 4 将三者放在同一连接建立链路中。STUN 提供可复用的消息格式和地址观察能力；TURN 在该消息体系上扩展 Allocation、Permission 和中继传输；ICE 调用 STUN 事务检查由本地接口、STUN 和 TURN 产生的候选。三者并非并列替代方案。一个常见会话会同时产生 host、server-reflexive 和 relay 候选，优先检查预期成本较低的直接候选，并保留中继候选作为可靠性保障。

这一协同还说明“地址存在”和“路径可用”是两个判断。STUN 返回反射地址，只能表明客户端到服务器方向形成了映射；ICE connectivity check 才在具体候选对上验证通信。TURN 地址的获得也不等于最

终一定中继，只有包含该候选的候选对检查成功并被提名后，中继才成为选定路径。因此，应用不能根据候选类型列表直接推断最终媒体路径。

5 WebRTC 中的 NAT 穿越

Web 实时通信（Web Real-Time Communication, WebRTC）把上述机制集成在浏览器的 `RTCPeerConnection` 中。设浏览器 A 和 B 位于不同 NAT 后，应用同时部署信令服务器、STUN 服务器和 TURN 服务器。信令服务器交换会话描述、ICE credentials 与 candidates，但不必转发媒体；STUN/TURN 服务器参与候选获取和检查；浏览器内的 ICE agent 负责路径选择。WebRTC 传输规范要求实现支持 ICE、STUN 和 TURN，以适应不同网络条件 [7]。

图 7 用虚线表示应用层信令或备用路径，用实线表示控制交互，并突出可能采用的直接媒体路径。浏览器 A、B 分别访问 STUN 和 TURN，所得候选经信令服务器转交对方。信令服务器知道会话参数，却不必看到实时媒体内容；TURN 服务器只有在中继候选被选中时才持续承载媒体。实际 WebRTC 还会在选定路径上运行 DTLS、SRTP 等安全协议，但其报文细节不属于本文讨论范围。

传统 ICE 可以等待一批候选收集完成后再统一发送。Trickle ICE 允许候选生成后立即通过信令发送，对端可提前形成候选对并开始检查，从而减少等待全部候选的连接建立时间 [8]。候选到达顺序不代表最终优先级，后续出现的更优候选仍可能参与检查。浏览器通过 `icecandidate` 事件逐个向应用报告候选，并以候选收集结束事件表示当前 gathering 完成 [9]。

典型路径可以分为三类。同一局域网内，host-host 候选对可能直接成功；位于不同 NAT 后且设备允许打洞时，server-reflexive 候选参与的候选对可能成功；严格 NAT、防火墙或 UDP 受限时，ICE 可能选中包含 relay candidate 的候选对。应用还可以使用 relay-only

实验条件	Host	Srflx	Relay	传输协议	收集时间	完成状态
无 ICE 服务器 (3 次)	待实测	待实测	待实测	待实测	待实测	待实测
仅 STUN (3 次)	待实测	待实测	待实测	待实测	待实测	待实测
STUN + TURN (3 次)	待实测	待实测	待实测	待实测	待实测	待实测

表 6 不同 ICE 服务器配置下的候选收集结果
Candidate gathering under different ICE server configurations

策略限制只使用中继候选, 减少向远端暴露部分本地或公网地址, 但代价是所有业务数据都进入 TURN 基础设施 [9]。

浏览器网络发生明显变化时, 原有选定候选对可能失去可达性。例如终端从校园无线网络切换到移动热点, 主机地址、默认路由和 NAT 映射都会改变。应用可以触发 ICE restart, 生成新的 ICE credentials, 重新收集和交换候选, 再建立新的 checklist。该过程能够复用上层会话关系, 但仍需信令通道把新的参数交给对端 [6], [9]。因而, 信令服务在媒体直连后仍有维护价值; 它不仅参与初次建连, 也为网络切换后的重新协商提供通道。

6 NAT 穿越方案的工程权衡

6.1 时延与可靠性

Host 或 server-reflexive 直连通常减少一个应用层中继点, 因此往往具有较短路径和较低排队时延。不过, 网络跳数不能直接等同于端到端时延: 公网路由、跨运营商互联、拥塞和 TURN 机房位置都可能改变结果。一台靠近双方且网络质量稳定的 TURN 服务器, 有时会比质量较差的直连路由更稳定。ICE 的默认优先级倾向直接候选, 工程系统仍应依据实际检查和运行状态判断路径, 而不是仅按候选名称推断性能。

可靠性方面, host candidate 依赖局域网或公网接口直接可达, server-reflexive candidate 依赖两侧映射和过滤行为兼容, relay candidate 则主要要求客户端能访问 TURN 服务器。TURN 因而承担连接兜底作用, 但也形成新的容量和故障边界。实际部署常在多个地域设置 TURN 节点, 并同时支持 UDP、TCP 或 TLS 传输, 以应对只允许特定出口协议的网络。

6.2 服务器成本、安全与隐私

STUN 主要处理短小的 Binding 消息, 服务器保存状态和转发流量较少。TURN 需要转发完整业务数据, 双向实时视频会持续消耗公网带宽、文件描述符和加密计算资源。优先直连能够降低服务器成本; 配置 relay candidate 则提高了连接覆盖率。两者的取舍与业务对连接失败、启动时延和服务成本的容忍度有关。

安全上, TURN 必须启用认证、Permission、配额和速率限制, 避免成为开放中继 [5]。STUN 返回的公网地址只是网络观察结果, 不能作为用户身份凭据。NAT 穿越解决的是可达性, 不负责确认通信对象身份, 也不能替代媒体加密。隐私方面, host 和 server-reflexive candidates 可能暴露局部网络或公网地址信息; 现代浏览器可用 mDNS 名称隐藏部分主机地址, 而 relay-only 策略使远端主要看到中继地址。更强的地址隐藏会增加 TURN 使用率, 需要由应用按场景选择。

比较维度	Host/反射地址直连	TURN 中继
业务数据经过服务器	否	是
路径长度	通常较短	通常增加一个中继点
NAT 兼容性	依赖映射和过滤行为	对 NAT 行为要求较低
连接可靠性	受具体网络环境影响	通常较高, 但依赖服务器
服务器带宽成本	较低	较高
地址隐私	可能暴露端点地址	可减少直接地址暴露
典型用途	优先通信路径	受限网络中的保障路径

表 5 直连路径与 TURN 中继路径比较
Comparison of direct and TURN-relayed paths

表 5 反映的关系不是简单的优劣排序。直连强调路径效率 and 低服务器成本, 中继强调在复杂网络中的可用性。ICE 同时准备两类候选并通过检查选择, 正是为了在单次会话中处理这种权衡。对隐私要求高的场景可以主动牺牲部分路径效率, 普通实时通话则通常先尝试直连, 再保留中继候选作为保障。

7 轻量实验

7.1 实验目标与环境

本实验只验证 ICE 服务器配置对浏览器候选收集结果的影响, 不测试媒体吞吐量、端到端时延或互联网范围的连接成功率。测试使用 WebRTC 项目的 Trickle ICE 示例页面 [10]。STUN 与 TURN 服务由同一台临时 coturn 实例提供, 服务器按官方配置说明启用长期凭据、独立 realm、fingerprint 和限定的 UDP 中继端口范围 [11]。浏览器保持 IceTransports=all, 不申请摄像头和麦克风权限, 避免媒体设备差异影响候选收集。

实验在同一浏览器、同一 Windows 网络环境和同一日期完成。三组配置分别为不设置 ICE 服务器、仅设置 STUN、同时设置 STUN 和 UDP TURN。每组独立执行三次, 记录 host、srflx、relay 候选数量、传输协议、从点击 Gather candidates 到 gathering complete 的时间以及是否完成收集。完整公网地址和 TURN 密码不进入论文。

7.2 实验结果

表 6 将在三组浏览器实测完成后回填。判断候选类型时以页面输出中的 typ host、typ srflx 和 typ relay 为准; 同一网络接口可能同时产生 IPv4、IPv6 或 mDNS 形式候选, 因此候选数不应直接解释为可用路径数。仅 STUN 组出现 srflx 可证明浏览器从服务器获得了公网映射观察值; STUN 加 TURN 组出现 relay 则证明 Allocation 和中继候选收集成功。这些现象仍不能证明某候选对会在真实双端会话中被提名。

7.3 实验局限

单台终端和单一接入网络只反映当前 NAT、防火墙与浏览器实现。家庭宽带、校园网和移动网络可能产生不同候选,运营商级 NAT 还会改变映射层级。浏览器可能使用 mDNS 隐藏本地地址,候选数量也会受网卡、IPv6 和接口状态影响。候选收集完成仅说明本地 ICE agent 已获得一组候选;实验没有第二个 peer,因而不观察候选对优先级、连通性检查、提名和媒体传输。本文据此只比较配置与候选类型之间的关系,不给出连接成功率和性能结论。

8 结论与展望

NAT 通过转换地址和端口缓解 IPv4 地址压力,同时使 NAT 后端点缺少稳定的外部可达端点。UDP 打洞利用双方出站发送建立映射和过滤状态,但其成功受映射复用、过滤规则、防火墙和网络层级影响。STUN 提供反射地址和连通性检查消息,TURN 提供中继地址及业务数据转发,ICE 将主机、反射和中继候选组成候选对,经检查与提名得到最终路径。WebRTC 将这些机制与应用信令和 Trickle ICE 结合,使直连效率与中继可靠性能够在同一连接过程中共同考虑。

工程实现需要同时面对时延、连接覆盖率、服务器带宽、安全和地址隐私。直连通常是较经济的优先路径,TURN 则是受限网络中的必要保障。随着 IPv6 普及,端点可能重新获得全局可路由地址,但状态防火墙、隐私策略和异构网络仍会保留连接检查需求。后续工作可以比较不同接入网络下的选定候选对,研究网络切换后的路径恢复,并结合 TURN 节点地域分布分析实际时延和资源成本。

参考文献

- [1] F. Audet and C. Jennings, 《Network Address Translation (NAT) Behavioral Requirements for Unicast UDP》, technical report RFC4787, 1 月 2007. doi: 10.17487/RFC4787.
- [2] B. Ford, P. Srisuresh, 和 D. Kegel, 《State of Peer-to-Peer (P2P) Communication across Network Address Translators (NATs)》, technical report RFC5128, 3 月 2008. doi: 10.17487/RFC5128.
- [3] B. Ford, P. Srisuresh, 和 D. Kegel, 《Peer-to-Peer Communication Across Network Address Translators》, 收入 *Proceedings of the 2005 USENIX Annual Technical Conference*, USENIX Association, 2005, 页 179~192. [在线]. 载于: <https://www.usenix.org/conference/2005-usenix-annual-technical-conference/peer-peer-communication-across-network-address>
- [4] M. Petit-Huguenin, G. Salgueiro, J. Rosenberg, D. Wing, R. Mahy, 和 P. Matthews, 《Session Traversal Utilities for NAT (STUN)》, technical report RFC8489, 2 月 2020. doi: 10.17487/RFC8489.
- [5] T. Reddy, A. Johnston, P. Matthews, 和 J. Rosenberg, 《Traversal Using Relays around NAT (TURN): Relay Extensions to Session Traversal Utilities for NAT (STUN)》, technical report RFC8656, 2 月 2020. doi: 10.17487/RFC8656.
- [6] A. Keränen, C. Holmberg, 和 J. Rosenberg, 《Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal》, technical report RFC8445, 7 月 2018. doi: 10.17487/RFC8445.
- [7] C. Jennings, T. Hardie, 和 M. Westerlund, 《Transports for WebRTC》, technical report RFC8835, 1 月 2021. doi: 10.17487/RFC8835.
- [8] E. Iyov, J. Uberti, 和 P. Hancke, 《Trickle ICE: Incremental Provisioning of Candidates for the Interactive Connectivity Establishment (ICE) Protocol》, technical report RFC8838, 1 月 2021. doi: 10.17487/RFC8838.
- [9] World Wide Web Consortium, 《WebRTC: Real-Time Communication in Browsers》. 见于: 2026 年 7 月 14 日. [在线]. 载于: <https://www.w3.org/TR/webrtc/>
- [10] WebRTC project, 《Trickle ICE Sample》. 见于: 2026 年 7 月 14 日. [在线]. 载于: <https://webrtc.github.io/samples/src/content/peerconnection/trickle-ice/>
- [11] coturn contributors, 《coturn: TURN and STUN Server》. 见于: 2026 年 7 月 14 日. [在线]. 载于: <https://github.com/coturn/coturn>